

Choreography Saga Pattern to ASP.NET Core + Azure Service Bus

1. High-Level Mapping

Saga Concept **ASP.NET Core + Azure Service Bus**

Local Transaction EF Core transaction inside a microservice

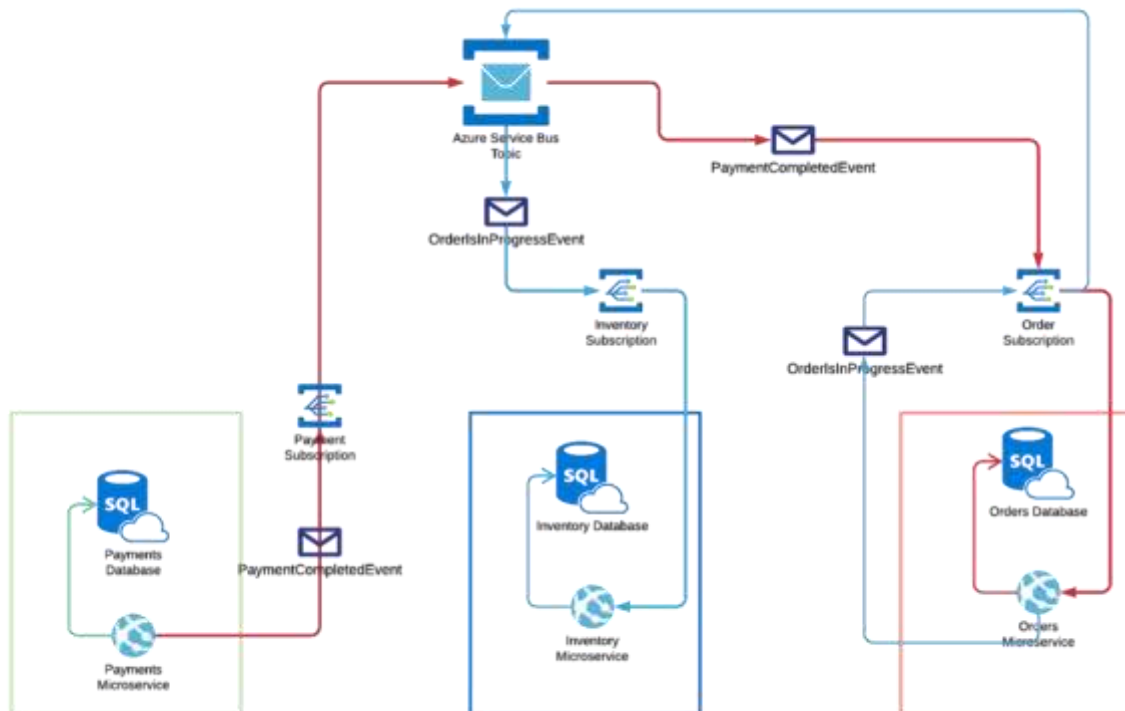
Event Azure Service Bus **Topic Message**

Event Publisher ASP.NET Core service publishing to topic

Event Subscriber Background Service / Message Handler

Compensation Reverse local transaction + publish event

No Coordinator No central API controlling the flow



2. Reference Architecture

Microservices

- **OrderService**
- **PaymentService**
- **InventoryService**

Each service:

- Is an **ASP.NET Core Web API**
- Has its **own database**
- Communicates **only via Azure Service Bus**

3. Azure Service Bus Design (Important)

Why Topics, not Queues?

Choreography requires **multiple subscribers** to the same event.

Topic Structure

Topic: order-saga-topic

Subscriptions:

- order-service-sub
- payment-service-sub
- inventory-service-sub

Each service:

- Publishes events to the topic
- Listens only to **relevant events**

This is the foundation of choreography.

4. Step-by-Step Flow (Mapped to ASP.NET Core)

Step 1: Order Service – Start the Saga

Controller Action

- Client calls POST /orders

OrderService

1. Saves order with status Pending
2. Publishes OrderCreated event to Service Bus

Conceptual Responsibility

- Starts the saga
- Does **not** call Payment or Inventory

Event Payload

```
{  
  "eventType": "OrderCreated",  
  "orderId": "123",  
  "amount": 5000  
}
```

Step 2: Payment Service – React to Event

PaymentService Background Listener

- Subscribes to OrderCreated

On receiving OrderCreated:

1. Processes payment
2. If successful → publish PaymentCompleted
3. If failed → publish PaymentFailed

Important

- PaymentService does not know OrderService internals

- Only reacts to the event
-

Step 3: Inventory Service – Continue the Saga

InventoryService Background Listener

- Subscribes to PaymentCompleted

On receiving PaymentCompleted:

1. Reserves stock
 2. If success → publish InventoryReserved
 3. If failure → publish InventoryFailed
-

Step 4: Order Service – Final State Update

OrderService Listener

- Subscribes to:
 - InventoryReserved
 - PaymentFailed
 - InventoryFailed

State Transitions

- InventoryReserved → Order = Completed
 - PaymentFailed → Order = Cancelled
 - InventoryFailed → Order = Cancelled (waits for compensation)
-

5. Compensation Flow (Failure Scenario)

Scenario: Inventory Reservation Fails

1. Inventory publishes InventoryFailed
2. PaymentService listens to InventoryFailed
3. PaymentService:

- Refunds payment
 - Publishes PaymentRefunded
4. OrderService:
- Marks order as Cancelled

Key Point

There is:

-  No rollback
 -  Only compensating actions
-

6. ASP.NET Core Implementation Mapping

Controllers

- Used **only for external requests**
- Example: CreateOrder

Background Services

Each service has:

- IHostedService or BackgroundService
- Responsible for:
 - Subscribing to Service Bus
 - Handling events

Message Handler Flow

Service Bus Message



Deserialize Event



Business Logic



Database Update



Publish Next Event

7. Event Versioning & Contracts

Best practice:

- Create a **Shared Contracts** project
- Contains event DTOs only

Example:

- OrderCreatedEvent
- PaymentCompletedEvent
- InventoryFailedEvent

This avoids tight coupling.

8. Correlation ID (Critical for Choreography)

Every event must include:

- OrderId
- CorrelationId

Purpose:

- Trace the saga across services
 - Debug failures
 - Enable distributed logging
-